2026
Software Requirements Specifications
Uni Textbook Marketplace

For **Demo 2**

Presented by:

NexusDev

Software Requirements Specifications (SRS)

Project: Uni Textbook Marketplace
Team: NexusDev
Client: Agile Bridge
University: University of Pretoria - COS 301 Software Engineering 2026
Document version: 3.0 - Demo 2
Last updated: 30 July 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

> For this document, we followed the incremental Agile approach. Sections are added and refined each sprint. Old versions of changed sections are moved to the Appendix.

Contents

1. Introduction

1.1 Business Need

University students across South Africa face significant challenges when trying to access affordable textbooks each semester. The current landscape for buying and selling used textbooks is fragmented and inefficient, and students rely on informal channels such as WhatsApp groups, Facebook Marketplace, and word of mouth. These platforms were never designed for academic textbook trading and suffer from three critical problems.

First, discovery is broken. Listings lack critical structured details such as edition, condition, and module code. A student looking for the fifth edition of a specific textbook for a specific module has no reliable way to filter for exactly what they need. They are forced to scroll through irrelevant listings and contact multiple sellers only to discover the edition is wrong.

Second, trust is absent. There is no mechanism to verify that a seller is a genuine university student. Privacy breaches are common; buyers and sellers exchange personal contact details with strangers before any agreement is reached, exposing both parties to risk.

Third, the process is misaligned with academic needs. No existing platform understands the academic calendar, module codes, or faculty structure. Students cannot browse by module, filter by semester, or find listings relevant to their specific course.

The Uni Textbook Marketplace directly addresses all three problems by providing a purpose-built, verified, module-aware platform for university students.

1.2 Platform Overview

The Uni Textbook Marketplace is a responsive web-based platform that enables verified university students to buy, sell, and swap second-hand textbooks in a safe, structured, and module-aware environment. Students register using their university email address, which is verified via a one-time password (OTP). Once verified, students can create structured textbook listings with details including ISBN, edition, condition, annotation level, module code, price, and supplementary notes. Buyers can browse listings filtered by faculty, module code, semester, edition, price range, and condition. An admin moderation layer ensures listing quality. All listings enter a pending state and are reviewed by an admin before becoming visible to other students. The platform is built for Agile Bridge and hosted on Microsoft Azure.

1.3 Scope

Delivered for Demo 1 (22 May 2026)

- Student registration with university email domain verification (OTP)
- Student login with JWT-based authentication
- Role-based access control (Student vs Admin)
- Structured textbook listing creation (UC1): ISBN, title, author, edition, condition, annotation level, module code, price, has notes
- Listing detail view for students (UC2): view own listings (approved + pending) and browse others' approved listings
- Admin listing review dashboard (UC3): approve or reject pending listings with soft delete and audit trail
- Module code search-as-you-type lookup, filterable by university
- Basic themes and form validation
- Responsive web interface (mobile + desktop)
- GitHub Actions CI/CD pipeline
- PostgreSQL database with full schema and seed data

Delivered for Demo 2 (31 July 2026)

- Privacy-first in-app messaging (UC4, Firebase Firestore microservice)
- Edit an existing listing while its status is PENDING or REJECTED (UC5)
- Smart search filters (UC6): price range, edition, condition, annotation level, module, faculty
- Saved searches (UC6 extension): a student can save a filter combination and revisit it later
- Wishlist (UC7): a student can save individual listings for later
- Admin audit log (UC8): a queryable, immutable record of every moderation action
- Staging environment with its own database, alongside production, with both auto-deployed from separate branches

Deferred to Demo 3

- **Notification system:** a cross-cutting event bus, notifications table, and bell-icon UI, originally scoped for Demo 2, moved to Demo 3 following a sprint re-scope.
- **Wanted board per module**
- **Pickup preferences** (on-campus spots, residences with location sharing)
- **Admin ban and report management tools**
- Meetup ratings (successful, no-show, cancelled)
- Swap mode (book-for-book or swap plus cash)
- Trust signals (verified badge, account age, listing history)
- Bundle deals per module

Explicitly out of scope

- **Payment processing:** the platform facilitates meetup-based exchanges only. No payment gateway will be integrated in any sprint.
- **Mobile application:** a React Native (Expo) mobile app is a stretch goal only, pursued after all core web requirements are delivered.
- **ISBN metadata API lookup:** manual entry only. External ISBN API integration is deferred.
- **Online textbook renting system:** identified as a potential future feature with legal considerations. Stubbed for now.
- **Multiple university support beyond the current three:** the allowlist covers `@tuks.co.za`, `@uct.ac.za`, and `@wits.ac.za`, with `@tuks.co.za` as the primary domain in active use. Support for further institutions is deferred.

1.4 Constraints

Constraint	Detail
Hosting	Provided by Agile Bridge on Microsoft Azure. The team does not control the infrastructure provider.
Authentication	OTP email verification is required. Students must verify their university email before accessing seller and messaging features.

Constraint	Detail
Email allowlist	Three domains are supported: `@tuks.co.za` (University of Pretoria, the primary domain in active use), `@uct.ac.za` (University of Cape Town), and `@wits.ac.za` (University of the Witwatersrand). The allowlist is configurable in the backend environment without a code change.
No payment processing	The MVP focuses entirely on meetup-based exchanges. No payment gateway, escrow, or financial transaction of any kind is included.
ISBN metadata	External ISBN lookup APIs are not integrated for Demo 1. All book details are entered manually by the seller.
No mobile app in MVP	The platform is a responsive web application only. React Native is a post-MVP stretch goal.
Firebase credentials	Firebase Firestore credentials are provisioned and in use. The messaging microservice (UC4) is live in both the production and staging environments.
Azure environment access	Both a production and a staging environment are live on Azure Container Apps, deployed automatically from `main` and `develop` respectively. See the SAS document for full deployment topology.

1.5 Definitions and Abbreviations

Term	Definition
OTP	One-Time Password. A 6-digit code sent to a student's university email to verify their identity during registration.
JWT	JSON Web Token. A signed, stateless token issued by the backend after successful login. It carries the user's ID and role claim.
UC	Use Case. A numbered interaction between an actor and the system.
UC1	Create a textbook listing (Student).
UC2	View listing details (Student).
UC3	Review listings (Admin).

Term	Definition
UC4	Send and receive in-app messages about a listing (Student).
UC5	Edit an existing listing while its status is PENDING or REJECTED (Student).
UC6	Filter listings with additional criteria, and save a filter as a saved search (Student).
UC7	Add or remove a listing from a personal wishlist (Student).
UC8	Query the audit log of moderation actions (Admin).
PENDING	The initial status of every new listing. It is not visible to other students until an admin approves it.
APPROVED	A listing that has been reviewed and approved by an admin. It is visible to all verified students.
REJECTED	A listing that has been rejected by an admin. It is soft-deleted and retained in the database with an audit log entry.
SOFT_DELETED	A listing that has been marked as deleted but is retained in the database. The deleted_at column is set.
FR	Functional Requirement.
SRS	Software Requirements Specification.
API	Application Programming Interface.
DTO	Data Transfer Object. A class used to define and validate the shape of request and response payloads.
RBAC	Role-Based Access Control. The system grants permissions based on a user's role (Student or Admin).
WCAG AA	Web Content Accessibility Guidelines Level AA. The accessibility standard the UI must meet.
UP	University of Pretoria.
CI/CD	Continuous Integration and Continuous Deployment. Automated pipelines triggered on every pull request.

2. User Stories / User Characteristics

2.1 Actors

The system has three actor types:

Verified Student (Buyer)
A registered and OTP-verified university student who uses the platform primarily to find and purchase or swap textbooks. They have a university email address from an allowlisted domain. They browse approved listings, filter by module code, faculty, edition, and condition, and contact sellers via the in-app messaging system.

Verified Student (Seller)
A registered and OTP-verified university student who creates structured textbook listings. They provide all required listing details including ISBN, edition, condition, annotation level, module code, and price. They monitor the status of their listings and are notified when an admin approves or rejects a submission.

Admin
A platform moderator who reviews all pending listings before they become visible to students. The admin can approve or reject listings, with rejection triggering a soft delete and an audit log entry. The admin also has full browsing access to all approved listings.

2.2 User Stories - Verified Student (Buyer)

US-B01
As a verified student buyer, I want to browse all approved textbook listings so that I can find a textbook I need without contacting sellers who do not have what I am looking for.

US-B02
As a verified student buyer, I want to filter listings by module code, faculty, edition, and condition so that I can quickly narrow down the results to exactly what I need for my specific course.

US-B03
As a verified student buyer, I want to view the full details of a listing including ISBN, edition, condition, annotation level, price, and whether notes are included so that I can make an informed decision before contacting the seller.

US-B04
As a verified student buyer, I want to see that every seller is a verified university student so that I can trust that the person I am dealing with is a genuine member of the university community.

US-B05
As a verified student buyer, I want the platform to only show me listings relevant to my university so that I am not exposed to textbooks from institutions using different editions or curricula.

2.3 User Stories - Verified Student (Seller)

US-S01
As a verified student seller, I want to create a structured textbook listing with fields for ISBN, title, author, edition, condition, annotation level, module code, price, and whether notes are included so that buyers can find exactly what they need without having to ask clarifying questions.

US-S02
As a verified student seller, I want to see my listings that are pending admin review, which are clearly marked with a "Pending Review" badge so that I know my listing has been submitted successfully and understand why it is not yet visible to other students.

US-S03
As a verified student seller, I want to indicate whether my listing includes supplementary study notes so that buyers who need additional study materials can identify and prioritise my listing.

US-S04
As a verified student seller, I want to view all my active approved listings in one place so that I can track what I currently have listed and manage my inventory.

2.4 User Stories - Admin

US-A01
As an admin, I want to see a dashboard of all listings currently in PENDING status so that I can review new submissions and ensure they meet platform standards before they become visible to students.

US-A02
As an admin, I want to approve a pending listing with a single action so that verified, accurate listings become available to buyers as quickly as possible without unnecessary delays.

US-A03
As an admin, I want to reject a pending listing with an optional note explaining the reason so that the seller understands why their listing was not approved and can correct the issue and resubmit.

US-A04

As an admin, I want rejected listings to be soft-deleted and retained in the database with a `deleted_at` timestamp and audit log entry so that there is a complete and recoverable record of all moderation actions for accountability purposes.

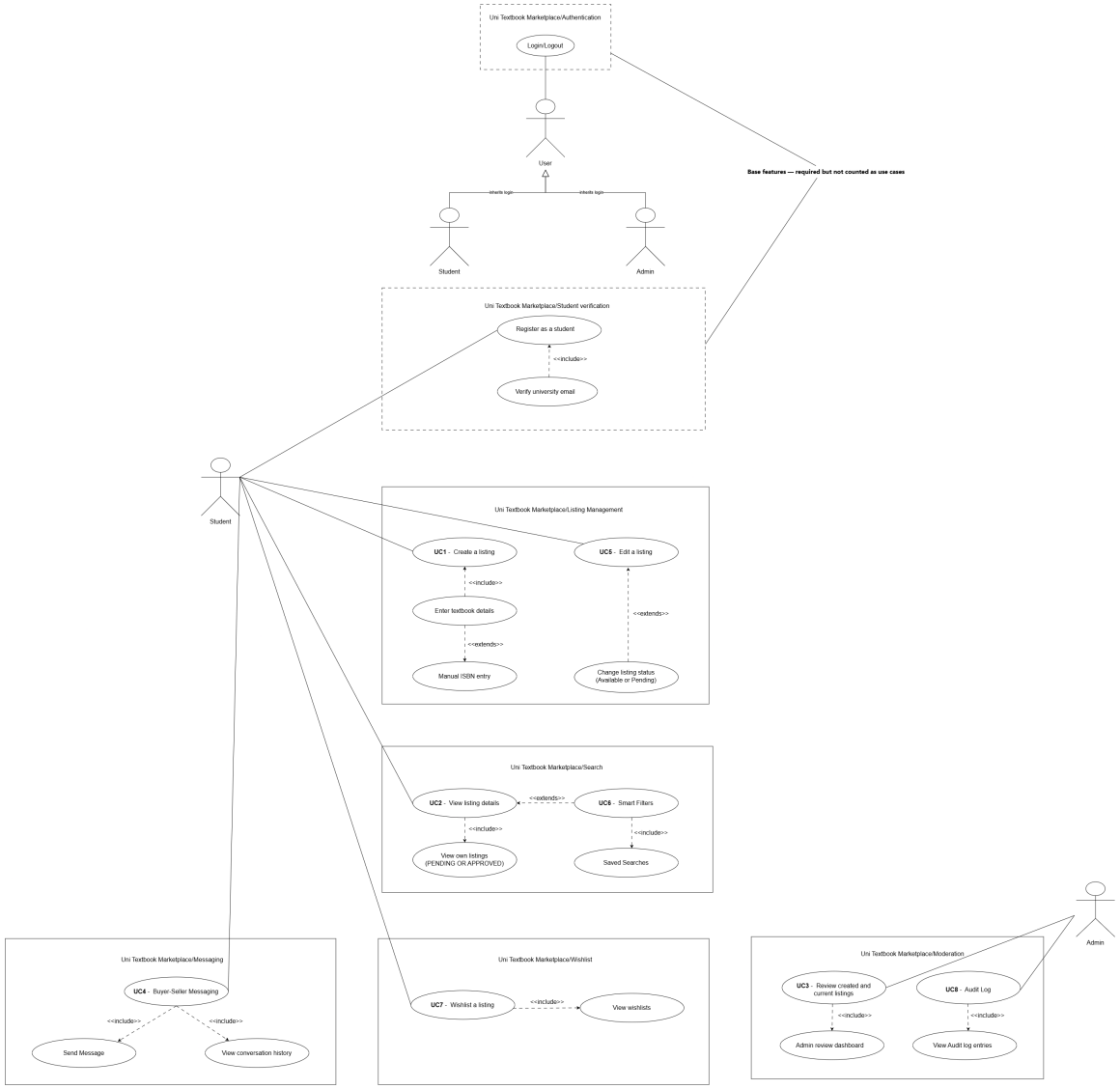
US-A05

As an admin, I want to browse all approved listings the same way a student can so that I can monitor listing quality and identify any listings that should be removed after approval.

3. Use Cases + Use Case Diagrams

3.1 Actor Hierarchy

The system defines the following actor hierarchy for use in use case diagrams. The base actor is **User**, which is specialised into **Verified Student** and **Admin**.



3.2 Use Case Overview

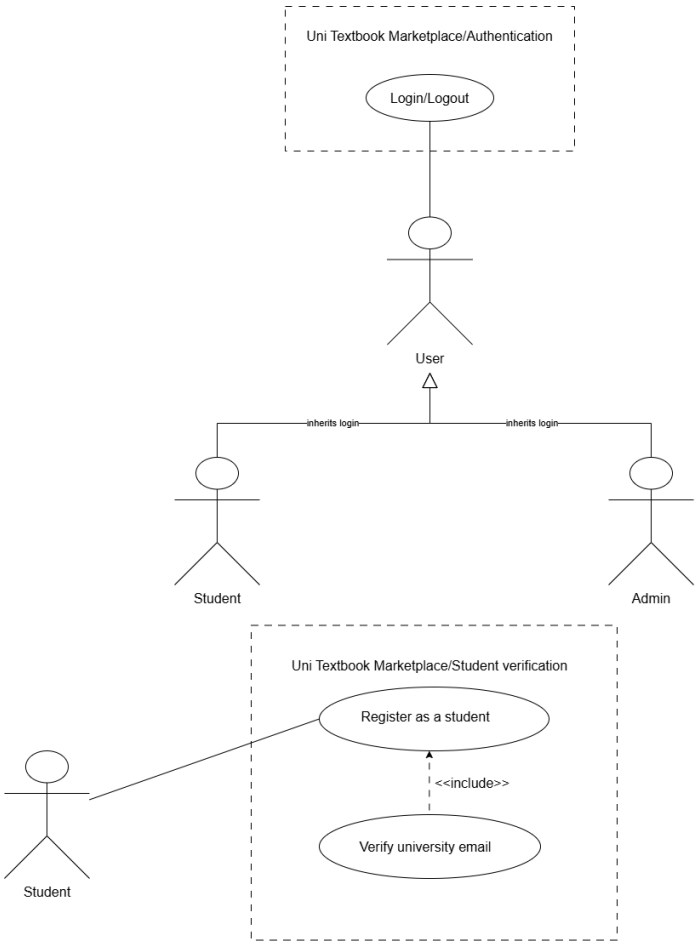
Registration and Login are base features and do not count toward the required use cases. UC1 to UC3 were signed off for Demo 1. UC4 to UC8 are new for Demo 2.

ID	Name	Primary Actor
UC0	Authentication (Register + Login + OTP)	Unverified Student (Base feature)

ID	Name	Primary Actor
UC1	Create a Textbook Listing	Verified Student (Seller)
UC2	View Listing Details	Verified Student
UC3	Review Listings	Admin
UC4	In-App Messaging	Verified Student
UC5	Edit or Withdraw a Listing	Verified Student (Seller)
UC6	Smart Filters and Saved Searches	Verified Student (Buyer)
UC7	Wishlist	Verified Student (Buyer)
UC8	Audit Log Query	Admin

3.3 UC0 - Authentication (Base Feature)

High-Level Description (TUCBW / TUCEW)



This use case begins when an unverified student navigates to the registration page and begins the multi-step registration process.

This use case ends when the student has successfully set a password, received a JWT, and is redirected to the listings page as an authenticated, verified user.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to Registration page.	System renders Step 1 (personal details: full name, surname).
2	Student enters full name and surname, clicks Next.	System validates non-empty fields and advances to Step 2.
3	Student enters university name and university email on Step 2.	System validates email domain against the allowlist (`@tuks.co.za`, `@uct.ac.za`, `@wits.ac.za`).
4	Student submits Step 2.	System generates a 6-digit OTP, stores it in the otps table with a 10-minute expiry, and sends it to the student's university email. System advances to Step 3.
5	Student enters the 6-digit OTP received by email.	System checks the OTP value and expiry again.
6	Student clicks Verify.	System marks the OTP as used, marks the user as verified (`is_verified = true`), and advances to Step 4.
7	Student enters a password and confirms it, then checks the Terms of Service checkbox.	System validates password length (minimum 8 characters) and that both passwords match.
8	Student clicks Register.	System hashes the password, saves the user record to the users table, issues a signed JWT with the student role claim, and redirects to the listings page.

Alternative Flows

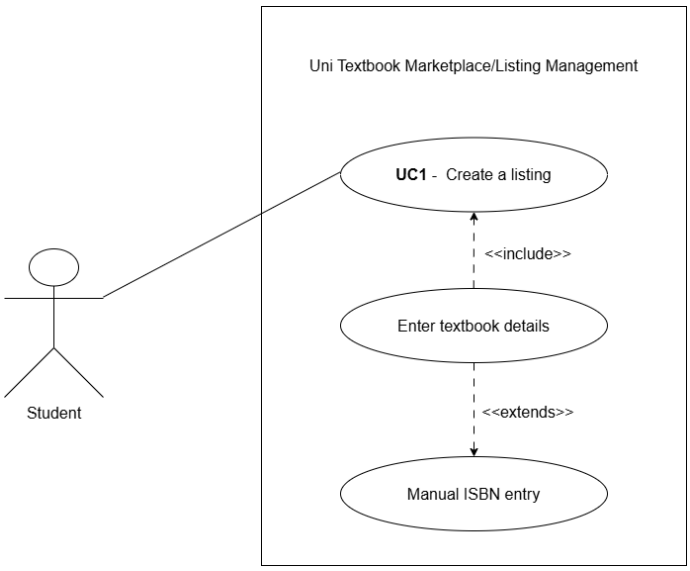
A1 - Invalid email domain: At step 3, if the email domain is not in the allowlist, the system displays an error stating that the email must end in one of the supported domains (@tuks.co.za, @uct.ac.za, or @wits.ac.za) and does not advance to Step 3.

A2 - Incorrect OTP: At step 6, if the OTP value does not match or has expired, the system displays "Invalid or expired OTP" and allows the student to request a new code.

A3 - Passwords do not match: At step 7, if the two password fields do not match, the system displays "Passwords do not match" and does not submit.

3.4 UC1 - Create a Textbook Listing

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified, authenticated student navigates to the Create Listing page and begins filling in the listing form.

This use case ends when the student submits the form and the system creates a new listing record with PENDING status and displays a confirmation with a "Pending Review" badge.

Expanded Use Case Table

Step	Actor Action	System Response
1	Authenticated student navigates to Create Listing.	System renders the listing form. The JWT is validated by the JwtAuthGuard.

Step	Actor Action	System Response
2	Student enters ISBN.	System accepts the value. ISBN lookup is manual for Demo 1.
3	Student enters title, author, and edition.	System validates that title and edition are non-empty.
4	Student selects condition from dropdown (new, good, fair, poor).	System stores the selected value.
5	Student selects annotation level (none, light, heavy).	System stores the selected value.
6	Student begins typing a module code in the module search field.	System queries GET /modules?search=[query]&university=UP and displays matching module codes as a dropdown.
7	Student selects a module from the dropdown.	System stores the module_id foreign key.
8	Student enters price in Rands.	System validates that price is a positive decimal value.
9	Student optionally toggles has_notes.	System stores the boolean value.
10	Student optionally uploads one or more photos.	System stores the photo URLs in the photo_url as array.
11	Student clicks Submit.	System sends POST /listings with the JWT in the Authorization header. Backend validates the DTO, creates the listing record with status = PENDING, and returns the new listing ID.
12		System displays a success message: "Your listing has been submitted and is awaiting admin review." The listing is visible on the student's My Listings page with a PENDING badge.

Alternative Flows

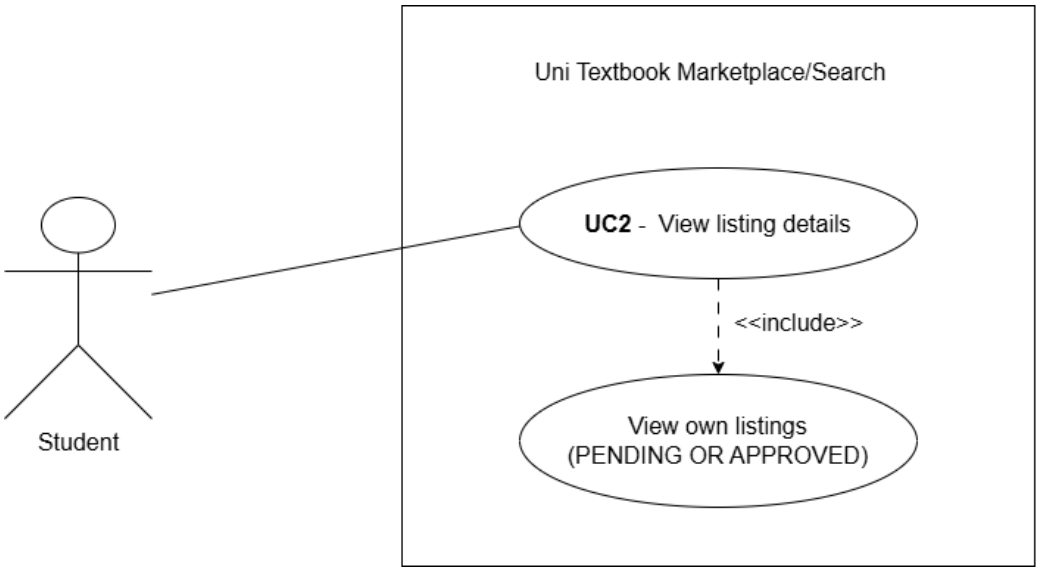
A1 - Not authenticated: At step 1, if the JWT is absent or expired, the JwtAuthGuard returns 401 and the frontend redirects to the login page.

A2 - Missing required fields: At step 11, if any required field (title, edition, condition, module, price) is empty, the system returns a 400 Bad Request with field-level validation errors. The form highlights the invalid fields.

A3 - Invalid price: At step 8, if the price is zero or negative, the system displays "Price must be greater than zero."

3.5 UC2 - View Listing Details

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified, authenticated student navigates to the listings page or their personal My Listings page.

This use case ends when the student has viewed the full structured detail page of a specific listing, including all book metadata, price, condition, annotation level, and listing status.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to Browse Listings.	System calls GET /listings, which returns all APPROVED listings. Each listing card shows title, edition, price, condition, and module code.

Step	Actor Action	System Response
2	Student optionally applies filters (module code, faculty, price range, condition).	System re-queries with filter parameters and updates the listing grid.
3	Student clicks a listing card.	System calls GET /listings/:id. The backend returns the full listing object including all fields.
4		System renders the listing detail page showing: title, author, ISBN, edition, condition, annotation level, has_notes, module code, price, photo carousel, and seller's verified badge.
5	Student navigates to My Listings.	System calls GET /listings/mine. The backend uses the JWT to identify the seller and returns both APPROVED and PENDING listings belonging to that student.
6		System renders the My Listings page. PENDING listings display a "Pending Review" badge. APPROVED listings display an "Approved" badge.

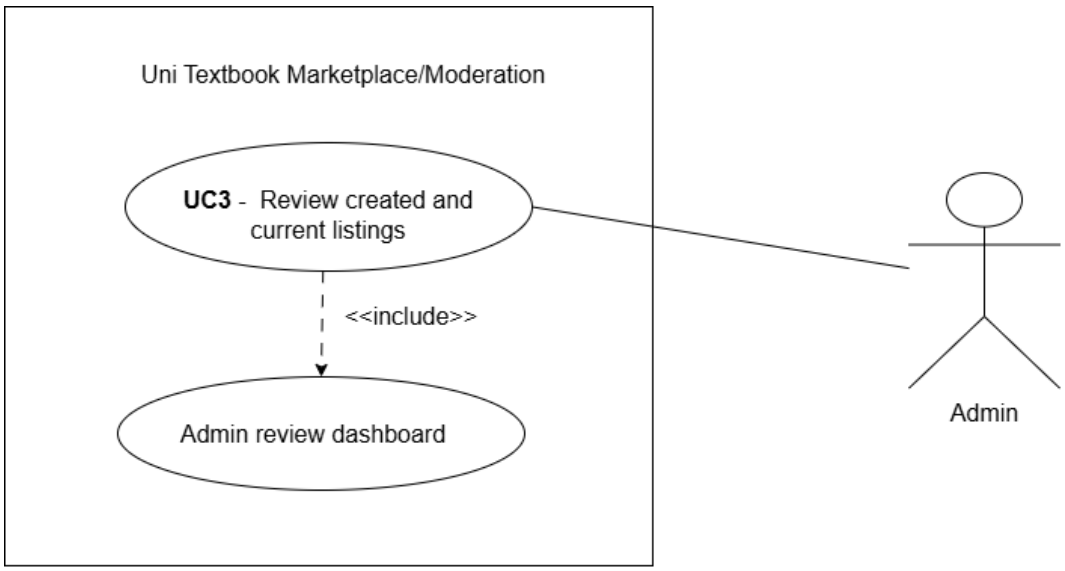
Alternative Flows

A1 - Listing not found: At step 3, if the listing ID does not exist or belongs to a REJECTED or SOFT_DELETED listing, the system returns 404 and displays "Listing not found."

A2 - Unauthenticated access: At step 1, if the JWT is absent, the frontend redirects to login. The browse endpoint requires authentication.

3.6 UC3 - Review Listings (Admin)

High-Level Description (TUCBW / TUCEW)

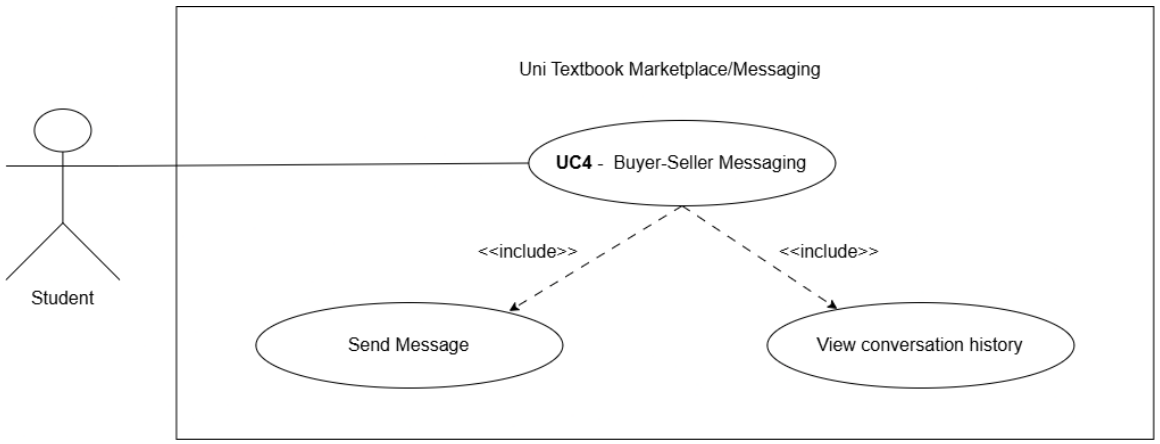


This use case begins when an authenticated admin navigates to the Admin Review Dashboard and sees the list of all PENDING listings.

This use case ends when the admin has either approved a listing (setting its status to APPROVED and making it visible to students) or rejected a listing (soft-deleting it and recording an audit log entry).

3.7 UC4 - In-App Messaging

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, viewing a listing, opens a conversation with the seller.

This use case ends when the two students have exchanged messages about the listing, without either party's personal contact details being shared outside the platform.

Messaging is handled by a separate Firebase Firestore microservice rather than the core NestJS backend, so that a Firestore outage cannot affect listing browsing, creation, or moderation. See the SAS document for the architectural rationale.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student views a listing detail page and clicks "Message Seller."	System validates the JWT (verified student) and renders the conversation panel.
2		Frontend authenticates directly against Firebase Firestore using the client SDK, scoped to conversations the student is a participant in, independent of the core NestJS backend.
3		If no conversation document already exists between this student and the seller for this listing, Firestore creates a new conversation document.

Step	Actor Action	System Response
4	Student types a message and clicks Send.	Frontend writes the message document directly to the conversation's Firestore collection, storing sender_id, listing_id, message text, and a timestamp.
5		Firestore's real-time listener pushes the new message to the seller's client if they have the conversation open.
6	Seller reads the message and replies.	The reply is written to the same Firestore collection and pushed back to the student in real time.
7	Student or seller reopens the conversation later.	Frontend queries Firestore for all messages tied to the conversation ID, ordered by timestamp, and renders the full history.

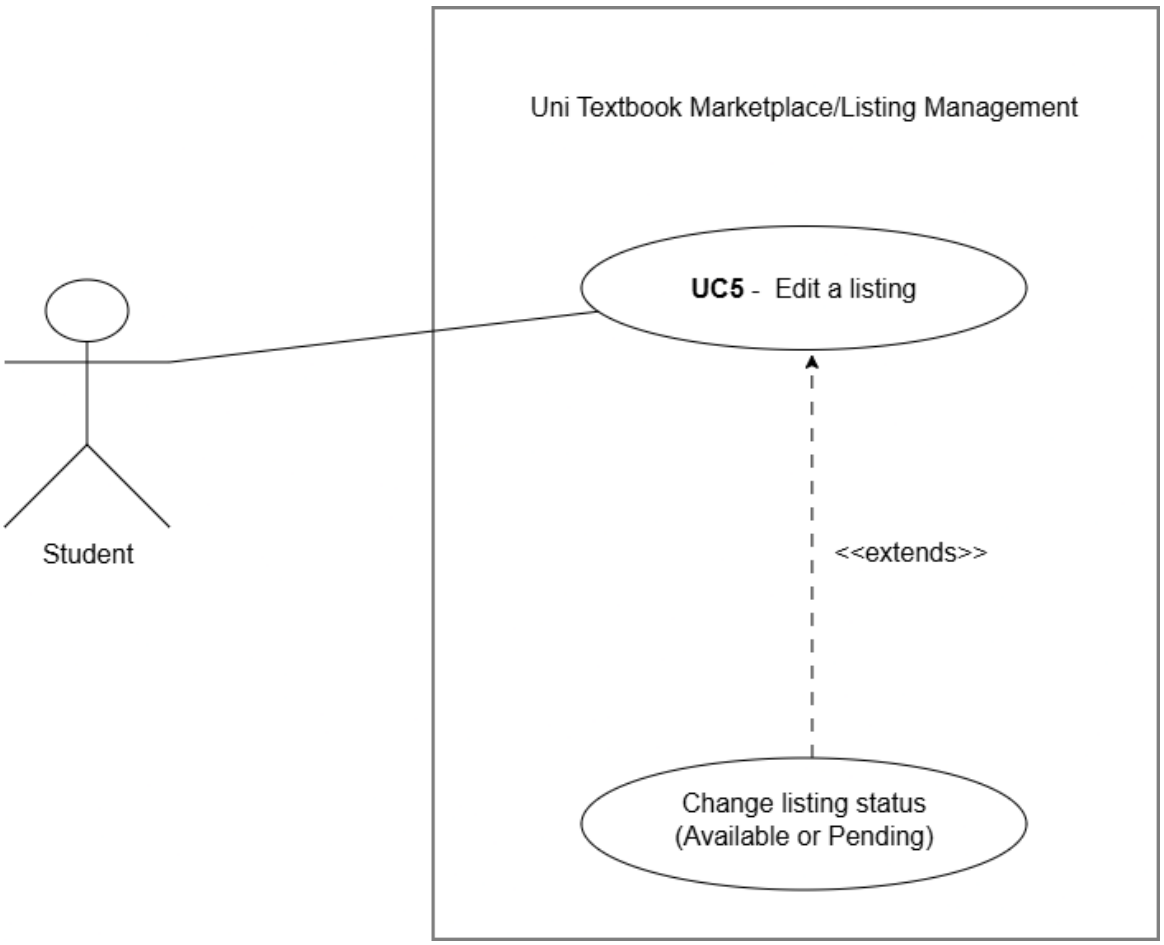
Alternative Flows

A1 - Firestore unavailable: If the client SDK cannot establish a connection to Firestore, the system displays "Messaging is temporarily unavailable." Because messaging is isolated in its own microservice, listing browsing, creation, and moderation remain fully available.

A2 - Not authenticated: If the student's JWT is absent or expired at step 1, the frontend redirects to the login page before the conversation panel is rendered.

3.8 UC5 - Edit a Listing

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, viewing one of their own listings on the My Listings page, chooses to either edit its details.

This use case ends when either the listing's details are updated and saved, as well as the listing's `listing_status` is set to `PENDING` to be reviewed by admin again.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to My Listings.	System calls GET /listings/mine and renders each listing with Edit and Withdraw actions.
2	Student clicks Edit on a listing.	System checks the listing's status. Editing is only permitted while status is `PENDING` or `REJECTED`; the Edit action is disabled for `APPROVED` listings.

Step	Actor Action	System Response
3		System renders the edit form pre-filled with the listing's current details.
4	Student updates one or more fields (price, condition, annotation level, module, description, photos).	System validates the updated fields using the same rules as listing creation.
5	Student clicks Save.	System sends the update to the backend. If the listing's prior status was `REJECTED`, the system resets it to `PENDING` for re-review. If it was `PENDING`, it remains `PENDING`.
6		System confirms the update and displays the listing's current status badge.

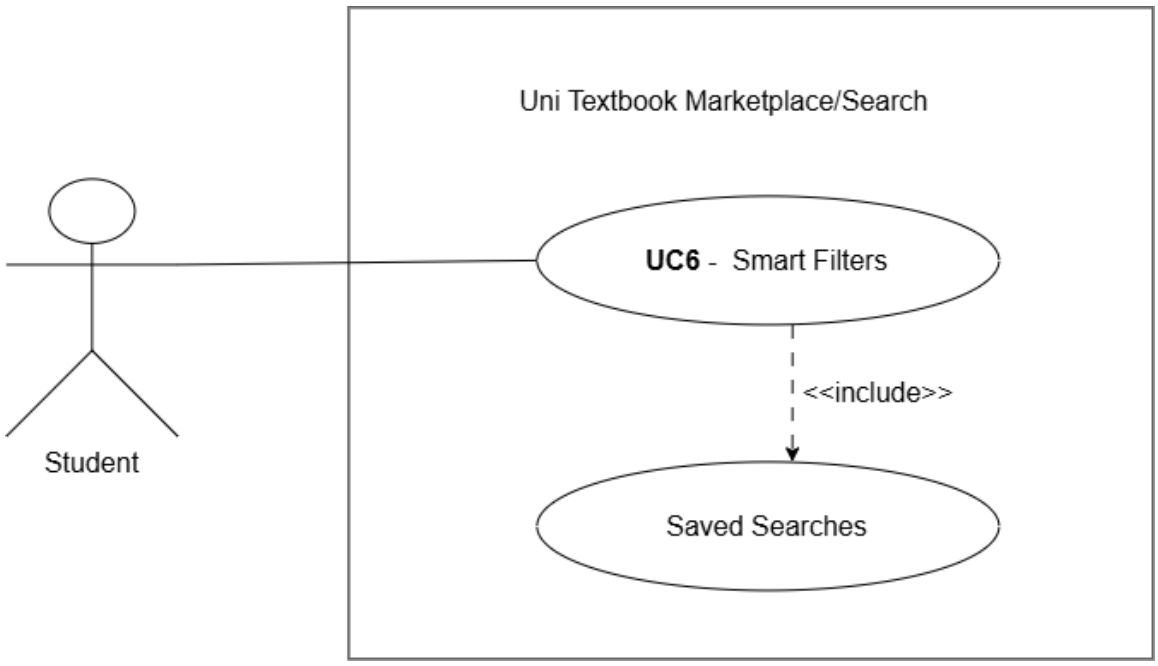
Alternative Flows

A1 - Attempt to edit an APPROVED listing: At step 2, the Edit action is not available for `APPROVED` listings; if attempted directly against the API, the backend returns 403 Forbidden.

A2 - Student cancels edit: At step 4, if the student navigates away without clicking Save, no changes are persisted.

3.9 UC6 - Smart Filters and Saved Searches

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, browsing listings, applies one or more additional filters beyond the Demo 1 module/faculty filter (price range, edition, condition, annotation level).

This use case ends when either the filtered results are displayed, or, if the student chooses to save the current filter combination, a saved search record is created that the student can revisit later without re-entering the same filters.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student on the Browse Listings page opens the extended filter panel.	System renders additional filter controls for price range, edition, condition, and annotation level, alongside the existing module/faculty filters.
2	Student sets a minimum and maximum price.	System stores the selected range as pending filter parameters.
3	Student selects an edition, condition, and/or annotation level.	System stores the selected values as pending filter parameters.

Step	Actor Action	System Response
4	Student clicks Apply Filters.	System calls GET /listings with the combined filter parameters and updates the listing grid with matching results.
5	Student clicks Save Search on the current filter combination.	System prompts for an optional label for the saved search.
6	Student confirms.	System stores the filter combination as a `filter_json` object tied to the student's user ID in the saved_searches table.
7	Student later navigates to Saved Searches.	System calls GET /saved-searches and returns the student's list of saved filter combinations.
8	Student selects a saved search.	System re-applies the stored filter parameters and re-queries GET /listings, updating the grid accordingly.

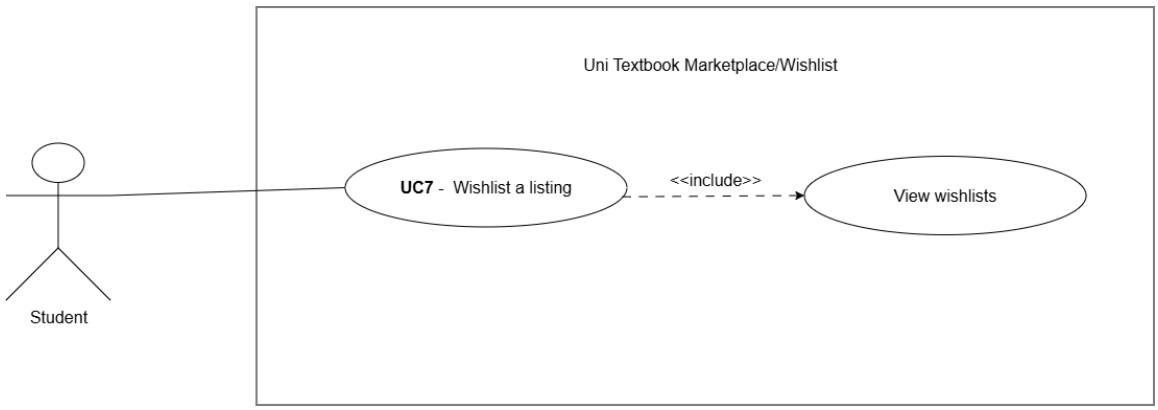
Alternative Flows

A1 - No listings match the filters: At step 4, if no listings satisfy the combined filter parameters, the system displays "No listings match your filters" and suggests clearing one or more filters.

A2 - Save attempted with no filters applied: At step 5, if no filter parameters have been set, the system disables the Save Search action.

3.10 UC7 - Wishlist

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, viewing a listing they are interested in but not ready to contact the seller about yet, chooses to save it to their wishlist.

This use case ends when the listing appears on the student's personal wishlist page, where it can also be removed.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student views a listing detail page and clicks the wishlist (bookmark) icon.	System sends POST /wishlist with the listing ID and the JWT in the Authorization header.
2		Backend creates a wishlist entry using a composite primary key of user_id and listing_id.
3		System updates the icon to a "saved" state on the listing card and detail page.
4	Student navigates to their Wishlist page.	System calls GET /wishlist/mine, which returns the student's wishlist entries joined with their listing details.
5		System renders the wishlist page showing each saved listing's title, price, condition, and status.
6	Student clicks the remove icon on a wishlisted listing.	System sends DELETE /wishlist/:listingId.

Step	Actor Action	System Response
7		Backend removes the wishlist entry and the listing disappears from the page.

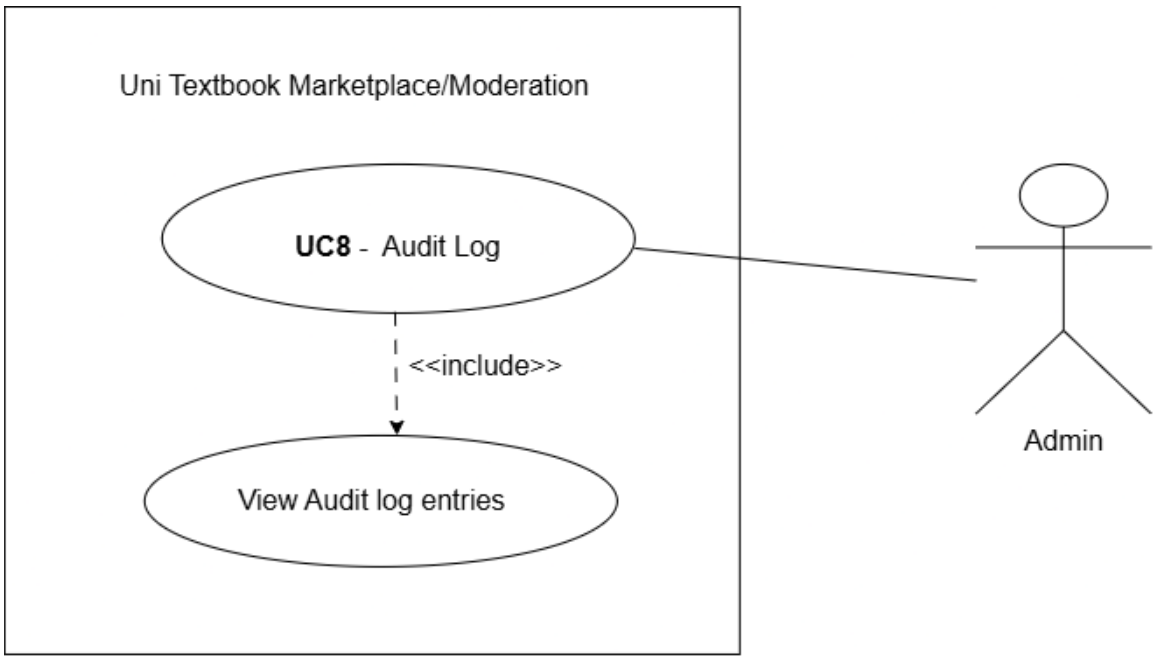
Alternative Flows

A1 - Listing already wishlisted: At step 1, if the listing is already on the student's wishlist, the icon instead reflects the "saved" state and clicking it removes the entry.

A2 - Wishlisted listing is later deleted: If the underlying listing is deleted, the wishlist entry is automatically removed via `ON DELETE CASCADE` on the listing foreign key, and the listing no longer appears on the student's wishlist page.

3.11 UC8 - Audit Log Query

High-Level Description (TUCBW / TUCEW)



This use case begins when an authenticated admin navigates to the audit log view to review past moderation actions.

This use case ends when the admin has retrieved either a full list of audit entries, summary statistics, or the audit history for a specific listing.

Expanded Use Case Table

Step	Actor Action	System Response
1	Admin navigates to the Audit Log page.	System calls GET /audit-log. The JwtAuthGuard validates the JWT and confirms the admin role claim.
2		System renders the full list of audit entries, showing entity_type, entity_id, action (APPROVED or REJECTED), performed_by, performed_at, and notes.
3	Admin switches to the Summary Statistics view.	System calls GET /audit-log/summary and returns aggregate counts of actions taken.
4		System renders the summary statistics (e.g. total approvals vs. rejections) as a table or chart.
5	Admin enters an entity type and entity ID to look up a specific listing's history.	System calls GET /audit-log/entity/:type/:id.
6		System returns and renders the full chronological audit history for that specific entity.

Alternative Flows

A1 - Non-admin access attempt: At step 1, if the authenticated user does not hold the admin role, the backend returns 403 Forbidden and the frontend does not render the audit log view.

A2 - No history for entity: At step 5, if no audit entries exist for the given entity type and ID, the system displays "No audit history found for this listing."

4. Functional Requirements

4.1 AuthService (UC0)

R1: Authentication

R1.1: Student Registration

R1.1.1 The system shall allow a student with a valid university email address to register by providing their full name, surname, university name, and email.

R1.1.2 The system shall reject any email address whose domain is not in the configured allowlist.

R1.2: OTP Generation and Verification

R1.2.1 The system shall generate a cryptographically random 6-digit OTP upon registration and send it to the student's university email address.

R1.2.2 The OTP shall expire after 10 minutes.

R1.2.3 The system shall mark the OTP as used after successful verification to prevent reuse.

R1.3: Login and JWT Issuance

R1.3.1 The system shall authenticate a registered, verified student by validating their email and hashed password.

R1.3.2 On success, the system shall issue a signed JWT containing the user's UUID and role claim (student or admin), to be used on all subsequent authenticated requests.

R1.4: Email Domain Allowlist

R1.4.1 The system shall enforce an email domain allowlist. Three domains are currently permitted: `@tuks.co.za` (University of Pretoria, the primary domain in active use), `@uct.ac.za` (University of Cape Town), and `@wits.ac.za` (University of the Witwatersrand).

R1.4.2 The allowlist shall be configurable in the backend environment variables without requiring a code change.

R1.5: Role-Based Access Control

R1.5.1 The system shall enforce role-based access control on all protected endpoints using a JWT guard and a role guard.

R1.5.2 Endpoints designated for Admin only shall return 403 Forbidden if accessed with a Student JWT.

4.2 ListingService (UC1)

R2: Listing Management

R2.1: Create Listing

R2.1.1 The system shall allow a verified, authenticated student to create a textbook listing by submitting a DTO containing: ISBN, title, author, edition (integer), condition (new/good/fair/poor), annotation_level (none/light/heavy), module_id (foreign key), price (positive decimal), has_notes (boolean), and photo_urls (array of strings).

R2.1.2 The listing shall be created with status = PENDING.

R2.2: Module Code Search

R2.2.1 The system shall provide a search-as-you-type module code endpoint accepting a search string and an optional university filter.

R2.2.2 The response shall include module code, module name, and faculty.

R2.3: Browse Approved Listings

R2.3.1 The system shall return all listings with status = APPROVED when a verified student calls the browse endpoint.

R2.3.2 Soft-deleted, rejected, and pending listings shall not be returned.

R2.4: View My Listings

R2.4.1 The system shall return all listings belonging to the authenticated student, including both APPROVED and PENDING listings.

R2.4.2 The response shall include the listing status so the frontend can render appropriate badges.

R2.5: View Listing Detail

R2.5.1 The system shall return the full listing detail object for a given listing ID.

R2.5.2 If the listing is REJECTED or SOFT_DELETED, the system shall return 404 Not Found. If the listing is PENDING, it shall only be accessible to its owner or an admin.

4.3 ModulesService (UC2)

R3: Module Management

R3.1: Module Search Endpoint

R3.1.1 The system shall expose a GET /modules endpoint that accepts a search query parameter and an optional university parameter.

R3.1.2 The endpoint shall perform a case-insensitive prefix search on module code and module name.

R3.2: Module Seed Data

R3.2.1 The system shall seed the database with a representative set of University of Pretoria module codes across all faculties. This is now fully implemented via an idempotent seed script, confirmed working against both a fresh and an existing database.

R3.2.2 The seed data shall include module code, module name, faculty, and semester.

4.4 ModerationService (UC3)

R4: Listing Moderation

R4.1: View Pending Listings

R4.1.1 The system shall return all listings with status = PENDING when an authenticated admin calls the pending listings endpoint.

R4.1.2 The response shall include full listing detail plus the seller's display name.

R4.2: Approve Listing

R4.2.1 The system shall allow an authenticated admin to approve a pending listing. On approval, the system shall set status = APPROVED, reviewed_by = admin

UUID, and reviewed_at = current timestamp.

R4.2.2 An audit log entry shall be written with action = APPROVED.

R4.3: Reject Listing with Soft Delete

R4.3.1 The system shall allow an authenticated admin to reject a pending listing with an optional rejection reason.

R4.3.2 On rejection, the system shall set status = REJECTED, deleted_at = current timestamp, reviewed_by = admin UUID, and reviewed_at = current timestamp, and create an audit log entry with action = REJECTED and the optional notes.

R4.3.3 The listing record shall be retained in the database (soft delete, not hard delete).

4.5 MessagingService (UC4)

R5: In-App Messaging

R5.1: Send and Receive Messages

R5.1.1 The system shall allow a verified student to open a conversation with the seller of a listing.

R5.1.2 Messages shall be delivered and stored through the Firebase Firestore microservice, independent of the core backend.

R5.1.3 The frontend shall connect to Firestore directly through the client SDK, rather than through the NestJS backend.

4.6 ListingService Extension (UC5)

R6: Edit and Withdraw Listing

R6.1: Edit Listing

R6.1.1 The system shall allow the owning student to edit the details of their own listing while their listing status is in `PENDING` or `REJECTED`

R6.1.2 Editing an `REJECTED` listing resets its status to `PENDING`.

4.7 SmartFilterService and SavedSearchService (UC6)

R7: Smart Filters and Saved Searches

R7.1: Extended Filtering

R7.1.1 The system shall allow a student to filter listings by price range, edition, condition, and annotation level, in addition to the module and faculty filters delivered for Demo 1.

R7.2: Save a Search

R7.2.1 The system shall allow a student to save their current filter combination as a saved search, stored as a JSON filter object tied to the student's user ID.

R7.2.2 The system shall allow a student to retrieve their saved searches.

4.8 WishlistService (UC7)

R8: Wishlist

R8.1: Add and Remove Wishlist Entries

R8.1.1 The system shall allow a student to add a listing to their personal wishlist.

R8.1.2 The system shall allow a student to remove a listing from their wishlist.

R8.1.3 A listing shall be removed from a student's wishlist automatically if the listing itself is deleted, since the wishlist entry references the listing by foreign key with `ON DELETE CASCADE`.

R8.2: View Wishlist

R8.2.1 The system shall allow a student to retrieve their full wishlist.

4.9 AuditLogService Extension (UC8)

R9: Audit Log Query

R9.1: Query Audit Log

R9.1.1 The system shall allow an authenticated admin to retrieve the full audit log.

R9.1.2 The system shall allow an authenticated admin to retrieve summary statistics over the audit log.

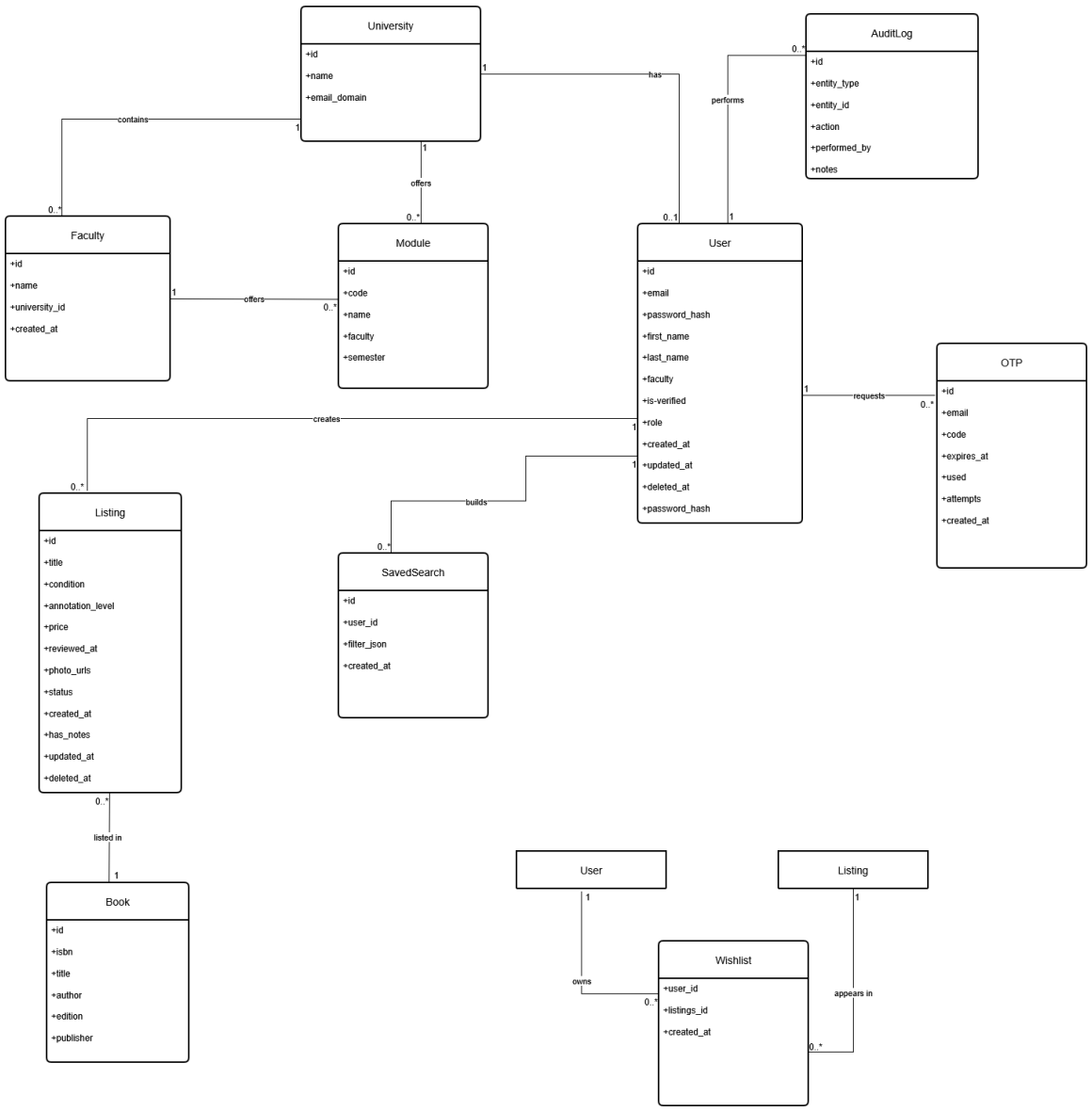
R9.1.3 The system shall allow an authenticated admin to retrieve the audit history for a specific entity, given its entity type and entity ID.

5. Non-Functional Requirements

Quality Attribute	Requirement and Quantification
Performance	Listing search and browse requests shall return within 2 seconds for at least 95% of requests under normal load.
Scalability	The stateless API layer shall support horizontal scaling without code changes, since no session state is held on the server.
Security	All protected endpoints shall enforce JWT-based authentication and role-based access control. Only OTP-verified students may create listings; only admins may moderate them.
Reliability	Core features (authentication, listings, moderation) shall remain fully available if the external messaging microservice (Firestore) is unavailable.
Maintainability	Backend test coverage shall be measured on every push and pull request via an automated CI pipeline running against a real ephemeral database, with results tracked through Codecov.
Accessibility	The UI shall meet WCAG AA compliance at 90% or above, measured through automated Lighthouse audits.

6. Domain Model

6.1 UML Diagram (Currently, as of Demo 2)



6.2 Entity Descriptions

University

Represents an institution registered on the platform. Stores the institution name and the email domain used for verification. A university has many faculties, and each user belongs to a university.

Faculty

Represents a faculty within a university (e.g. Engineering, Built Environment and IT). Stores the faculty name and a foreign key to the university it belongs to. A faculty offers

many modules, sitting between University and Module in the hierarchy.

User
Represents a registered platform participant. Stores identity information (name, email, password hash), verification status, role (student or admin), and faculty. A user can have many listings (as a seller) and many audit log entries (as an admin who performed moderation actions). A user's university_id is set to NULL if the university record is deleted (ON DELETE SET NULL) so the user account is preserved.

OTP
Represents a one-time password record created during registration. Stores the target email, the 6-digit code, the expiry timestamp, a used flag, and a creation timestamp. Each OTP is consumed after a single successful verification.

Module
Represents a university module (course). Stores the module code (e.g. COS301), module name, and semester, along with a foreign key to the faculty that offers it. A module can have many listings associated with it.

Book
Represents the physical book being listed. Stores ISBN, title, author, edition, and publisher. A book can appear in multiple listings (e.g. the same textbook listed by multiple sellers).

Listing
The central entity of the platform. Represents one student's offer to sell a specific book. A listing links a seller (User), a book (Book), and a module (Module). It stores price, condition, annotation level, photo_urls, and a status (PENDING, APPROVED, REJECTED, SOFT_DELETED). It also stores reviewed_by (admin UUID), reviewed_at, deleted_at, and created_at for a full lifecycle record.

AuditLog
Records every moderation action performed by an admin. Stores entity_type, entity_id (the listing UUID), action (APPROVED or REJECTED), performed_by (admin UUID), performed_at, and optional notes. Provides an immutable accountability trail.

SavedSearch
Represents a filter combination a student has chosen to save for later. Stores a foreign key to the owning User, a `filter_json` field holding the saved filter parameters, and a creation timestamp. A user can have many saved searches.

Wishlist
A join entity linking a User to a Listing they have saved for later. Uses a composite primary key of `user_id` and `listings_id`, both with `ON DELETE CASCADE`, so a wishlist

entry is automatically removed if either the user or the listing is deleted.

7. Appendix

This section holds previous versions of changed SRS sections. Sections are moved here when substantially updated, not deleted. This maintains a historical record of requirement evolution.

7.1 Previous Versions

Version 1.0 (Sprint 1)

Initial document structure with Introduction and User Stories. Moved to Appendix after Sprint 2 additions of use cases, functional requirements, API contracts, domain model, and architectural requirements.

Version 2.0 Functional Requirements (pre-realignment)

The original Sprint 2 draft numbered functional requirements with flat, service-prefixed IDs (e.g. `FR-AUTH-01`, `FR-LIST-02`, `FR-MOD-03`, `FR-MOD-S01`). These have been superseded by the hierarchical FR1 / FR1.1 / FR1.1.1 numbering scheme shown in Section 4, aligned with the requirement decomposition format taught in the Systems Engineering lecture (Ch. 3–4).

Sprint 2 Closure Notes (18 May 2026)

- Three use cases signed off: UC1 (Create listing), UC2 (View listing), UC3 (Review listings).
- Registration and login are base features and do not count toward the three use cases.
- Authentication method: OTP sent to university email.
- Email allowlist: `@toks.co.za` and `@up.ac.za` only.
- Module code search-as-you-type lookup, filterable by university.
- Listing lifecycle uses soft delete with audit log.
- Branding: follow Agile Bridge website (www2.agilebridge.co.za).
- Renting feature: stubbed for now due to legal considerations.
- Nice-to-haves (wanted board, swap mode, pickup preferences): not elevated for Demo 1.

Demo 2 Closure Notes (30 July 2026)

- Five new use cases delivered: UC4 (Messaging), UC5 (Edit/Withdraw Listing), UC6 (Smart Filters and Saved Searches), UC7 (Wishlist), UC8 (Audit Log Query).
- Architectural requirements, technology requirements, and API contracts moved out of this document into the separate SAS document, per the Demo 2 instructions.
- Non-Functional Requirements added as a dedicated section with five quantified quality attributes.
- Email allowlist corrected throughout this document. Three domains are actually supported, `@tuks.co.za` (primary), `@uct.ac.za`, and `@wits.ac.za`. `@up.ac.za` was never actually a supported domain in practice and has been removed everywhere it previously appeared.
- Notification-core system (event bus, notifications table, bell icon UI), originally scoped for Demo 2, moved to Demo 3.

Uni Textbook Marketplace

In collaboration **with:**

This document is version-controlled in the GitHub repository under `/docs/Demo 2/Software Requirements Specifications.pdf`